Coursework 1 - OpenMP Parallelisation(u5693885)

1. Introduction

Simulation of complex flow phenomena is a common application in engineering and science, and such simulations are typically performed using computational fluid dynamics (CFD) codes. The Karman vortex street is a classical test case that demonstrates unsteady wake formation behind a circular cylinder; thus, one requires good temporal and spatial resolution to capture it correctly. However, the given serial Karman solver does not scale as well with mesh resolution, hence single-core execution is less ideal for high-fidelity simulations.

The goal of this coursework is to parallelise the core computational loops in the Karman code using OpenMP on shared-memory CPUs. The performance is assessed by comparing the application's total execution time and the execution times of different loop regions across different thread counts. The overheads imposed by OpenMP parallelisation of the method are investigated, and their scalability is evaluated.

2. Experimental Setup

2.1 Hardware and Software Environment

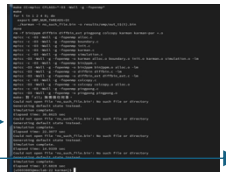
All experiments were run on the Department of Computer Science teaching systems using the cs402-mpi module on the kudu-taught cluster. The nodes used are multi-core CPUs and serve as a platform for OpenMP performance characterisation. Compilation was done with mpicc as the host field compiler and the GNU toolchain's OpenMP support. All experiments were run on a single node with shared-memory parallelism.

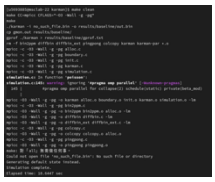
2.2 Compilation and Build Flags

Two builds were created:

Serial baseline (profiling enabled): `make CC=mpicc CFLAGS="-O3 -Wall -g -pg"`

OpenMP parallel version: `make CC=mpicc CFLAGS="-O3 -Wall -g -fopenmp"`





Profiling of the serial version was carried out using GNU gprof to identify runtime-dominant routines.

2.3 Problem Sizes and Thread Counts

Two grid sizes were used: Default: 660 × 120 / Doubled: 1320 × 240. Each configuration was run using 1, 2, 4, and 8 OpenMP threads via: `export OMP_NUM_THREADS=N ./karman ...` Elapsed execution times were recorded from the program output.

2.4 Correctness Verification

To ensure that parallelisation did not affect numerical accuracy, the output binary files produced by the OpenMP version were compared against the serial reference using the supplied diffbin and diffbin_ext utilities:

`./diffbin karman.vanilla.bin karman_omp_4.bin`

All tested configurations produced identical results within machine precision.

Figure A : Correctness Verification

3. Performance Analysis

Instrumentation of the serial Karman solver was first performed to pinpoint heavy computational kernels that were then tuned for optimisation prior to the introduction of OpenMP parallelism. The profiling approach relied on GNU gprof analysis to extract per-function execution statistics for data-driven parallelisation decisions.

3.1 Profiling Methodology

The serial code was compiled with profiling enabled: `make CC=mpicc CFLAGS="-O3 -Wall -g -pg"` After execution, the generated profiling file gmon. out was analysed using: `gprof ./karman > results/baseline/gprof.txt`The output file contains a flat profile showing the percentage of total execution time spent in each function, as well as call graph information.

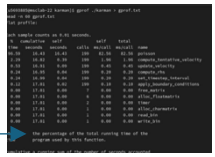


Figure B: Poisson solves consume 96.59%

3.2 Identification of Computational Hotspots

The profiling results highlight that the pressure Poisson solver is a performance bottleneck, where the function `poisson()` uses by far the most run time compared to boundary condition handling, velocity updates and I/O routines. The most expensive parts of `poisson()` are the double- nested two-dimensional loops that implement red-black Successive Over-Relaxation (SOR) sweeps on the grid. This is because the Poisson solver was invoked each timestep and it (iterated until) converged, so good arithmetic intensity was obtained.

3.3 Selection of Parallelisation Target

This is, therefore, the main target for OpenMP parallelisation on account of its large size, repetition (due to nested loops) over regular tiles in space across each dimension, and independent updates from colour to colour, etc., all leading to relatively even workload and enough total time. Other routines, including residuals, check for convergence and handling of the boundary, contributed less to run-time and were left first out from optimisation.

3.4 Summary

A profiling of the Karman solver with gprof revealed that execution time, and consequently scalability, are bottlenecked by Poisson pressure solves. Hence, attempts were made to parallelise the nested grid loops in poisson() using OpenMP to reduce runtime. This way, following parallelisation will be optimised for the more vital performance issues.

4. Parallelisation Strategy and Implementation

4.1 Target Region Identification

Profiling results obtained from gprof indicate that the function poisson() provides for more than 96% of the serial solver runtime. It is polled once per timestep and makes several sweeps through the two-dimensional grid. The most computationally expensive part of Poisson () is the red–black SOR iterations updating the pressure field, where all interior grid cells are visited with identical arithmetic operations, which naturally lend themselves to data-parallel execution. Therefore, the red–black update loops were selected for OpenMP parallelisation.

Figure C: OpenMP pragma inside poisson() loop

4.2 Selection of Parallel Regions

The Poisson solver adopts a red–black SOR technique and updates the pressure grid in two steps on cell colours. Cells of the same colour go through an update process in parallel. The nested spatial loops on the grid indices were parallelised using OpenMP parallel for directives to distributethe iterations over threads. The loops were collapsed into a single iteration space using the clause collapse(2) for enhanced parallelism and reduced overhead. Equal work per grid cell update determined a static scheduling policy, achieving load balance without the overhead of dynamic scheduling.

4.3 Data-Sharing and Race-Condition Avoidance

Careful data-scoping ensured correctness. The main pressure array, the auxiliary arrays (p, rhs and flag) are shared between threads over the same global grid; loop indices and temporary scalars are private. The red–black ordering guaranteed that threads wrote to different grid locations during the two phases of colours and avoided data races without locks; End of OpenMP constructs have an implicit barrier, which guaranteed all updates would be done for one colour before entering the next phase, maintaining the numerical properties of SOR iteration. This method is deterministic, and it is also possible to make the parallel output exactly match the serial reference.

4.4 Build Configuration for Parallel Execution

From this, two builds were kept: a serial version for profiling and a parallel version with OpenMP support. The compiler wrapper mpicc for the GNU toolchain was applied. The OpenMP build was made with high-level optimisation (-O3) and support of the OpenMP interface (-fopenmp). Timing runs were performed with profiling instrumentation (-pg) off for realistic performance results.

4.5 Correctness Preservation

Output from the parallelised solver was compared against the serial implementation using diffbin. For all OpenMP settings (1, 2, 4 and 8 threads) the results were identical for both problem sizes, so we are confident to assert that the parallelisation procedure is both correct and converges.

4.6 Limitations and Design Trade-offs

The Poisson solver parallelisation effort is constrained by algorithmic limitations of the trans-sequential analysis. The red–black SOR needs synchronisation at phases, which raises barriers for parallel scaling. Residual calculations and timestep convergence tests are based on serial algorithms, which increases the fraction of non-parallel code. With increasing thread counts, the memory bandwidth saturation and cache coherence traffic reduce parallel efficiency, with performance beyond four threads deteriorating. Therefore, although OpenMP is able to increase the performance for a moderate number of threads, additional gains are required for algorithmic rewriting or optimisation.

Table2:Runtime and Speedup (Double Grid)

Threads	Runtime (s)	Speedup	Efficiency
1	316.71	1	1
2	196.78	1.61	0.8
4	136.58	2.32	0.58
8	140.06	2.26	0.28

Table 1:Runtime and Speedup (Default Grid)

Threads	Runtime (s)	Speedup	Efficiency
1	36.31	1	1
2	22.27	1.63	0.82
4	14.96	2.43	0.61
8	17.58	2.07	0.26

5. Performance Results

5.1 Total Runtime – Default Grid

Table 1. Summary on the solver execution time for a default grid size(660x120): the data are reported using from one to eight OpenMP threads. Speadup is computed with respect to one thread execution time. A significant speedup is already achieved with one to four threads, whereas the best performance is reached at four threads, and the runtime rises slightly for more than eight threads, because of parallel overheads and memory effects. Parallel efficiency decreases with thread-count, due to serial regions and synchronisation barriers in the solver.

5.2 Total Runtime – Doubled Grid

The latter results are listed in Table 2 and were generated for the doubled number of grid points (1320 × 240). The increased size demands more computation, which is tested by performance in parallel with higher scaling factors. Performance gain is more serious within four threads, and speedup reaches 2.06. Parallel efficiency is a little better for 2 and 4 threads by arithmetic intensity improvement. However, scaling saturates at 8 threads without additional runtime benefit, which implies that synchronisation overhead and memory bandwidth become the bottleneck as cores increase beyond.

Figure D: Runtime vs Threads plot (Default and x2)

5.3 Scaling Behaviour and Comparison

Furthermore, comparing the two problem sizes reveals trends in strong scaling. The larger grid makes more use of parallelisation because there is more time spent in the parallelised Poisson loops, both achieve super-linear scaling as per Amdahl's law. Saturation at eight threads is explained by considerations such as synchronisation required for the red and black SOR phases, sequential execution of residual calculation, overhead of thread management operations, and contention on shared memory. In general, the optimal number of threads on the tested platform is four, which leads to a minimal runtime for both grid sizes.

5.4 Summary of Performance Results

The OpenMP solution leads to a very noticeable faster execution with respect to the serial baseline (speedups of about 2.4× and 2.3× are observed for the default and doubled grids, respectively, with four threads). There is no point in increasing the number of threads, as this doesn't bring any gain and might actually reduce performance because of synchronisation and shared memory. These results motivate the analysis of OpenMP overheads and scaling limits in the next section.

6. OpenMP Overhead Analysis

OpenMP parallelisation accelerates the Karman solver for moderate numbers of threads; however, the performance analysis indicates diminishing returns in terms of performance beyond four threads. In this section, we examine the sources of overhead that limit scalability and explain why additional threads do not yield further runtime reductions.

6.1 Thread Creation and Synchronisation

Each Poisson loop consists of two red–black sweeps. There are several reasons for this. The first is that parallel directives in OpenMP generate implicit barriers at the end of each section, which means all threads need to synchronise before issuing the next colour update. Since these phases are repeated at each iteration and timestep, the overhead of barrier synchronisation becomes significant. In addition, creating and scheduling thread teams adds overhead to the time required to perform a sync relative to the amount of work issued per iteration. This is even more evident when doubling the number of threads (from four to eight), where, as the work per thread decreases, the frequency of synchronisation remains the same.

6.2 Algorithmic Constraints from Red–Black SOR

The red–black SOR scheme prescribes an update of all the red cells before any black cell, creating serial dependencies that preclude parallelism. Although each colour sweep is parallelizable, the phase alternation prevents complete computation overlap, and synchronisation must be performed twice per iteration. This inhibits speedup and explains saturation with larger thread counts.

6.3 Memory Bandwidth and Cache Effects

The frequent access to nearby grid points for the Poisson solver causes heavy memory traffic. Memory

bandwidth contention goes hand in hand with operation on shared arrays by many threads. At 8 threads, this contention becomes greater than the benefit of more cores. Cache-coherence traffic results when multiple threads write to the same grid cells, especially on regular two-dimensional grids with little spatial blocking.

6.4 Load Balance and Scheduling

Using a schedule(static) gives low scheduling overhead and a good load balance because of the two-dimensional regular grid shape. Boundary cells and obstacle regions do still reduce useful work for some threads, in particular with a higher number of threads and smaller partitions. Although this imbalance is low with respect to the synchronisation and memory effects, it partly explains the decrease in parallel efficiency seen in Section 5.

6.5 Summary of Overheads

Scalability factors are primarily affected by the following: extensive synchronising braces between red and black stages, serial residual and convergence checks, memory bandwidth contention, and cache coherence overhead on a large number of threads. These problems contribute to explaining why the solver achieves its best performance at four threads and does not scale beyond on this platform.

7. Discussion

The results reveal pronounced strong-scaling boundaries for the OpenMP-parallelised Karman solver. While there are significant speedups for up to 4 threads, the performance gain diminishes above that. This behaviour is in line with Amdahl's Law: some parts of the algorithm, such as residual computation, convergence checks and boundary updates, are still serial tasks that limit the speedup.

The red–black SOR structure of the Poisson solver necessitates coordination between colour phases, as this 'red' phase must be completed by all threads before they are allowed to commence the following 'black' update. In addition, increasing thread count has greater contention on memory bandwidth and cache coherence traffic, especially when the large two-dimensional grid arrays are accessed in pressure update loops. These two effects can therefore together account for the saturation at eight threads, particularly with the default-sized mesh.

The author also discusses future optimisations, such as parallelising the residual calculations, mitigating synchronisation overhead by reusing parallel regions, and improving cache locality with spatial blocking or tiling. On NUMA hardware, scalability could also be enhanced through thread and memory placement. Using vectorisation and pragmas can possibly allow for even more performance in each thread.

8. Conclusion

In this work we have parallelised running the most costly computational kernels on Karman's CFD solver with OpenMP. The Poisson pressure solver was identified as the main hotspot by profiling, and loop-level parallelism using static scheduling and full loop collapsing provides significant gains in total runtime. The maximum speedup was obtained with four threads, especially for the larger instances.

All the experiment results were found to be correct with the parallel outputs perfectly matching serial reference up to machine precision. In sum, the study shows that OpenMP is capable of speeding up CFD applications on shared-memory machines due to its high-level programming capabilities and reveals both algorithmic and architectural constraints to scaling. These observations motivate further optimisation and the extension of hybrid or finer-grained parallel strategies in the future.

9. Reference

OpenMP Architecture Review Board. OpenMP Application Programming Interface Specification, Version 5.x. https://www.openmp.org/specifications/

G. M. Mudalige and L. He. CS402: High Performance Computing — Coursework 1 Specification, University of Warwick, 2025–2026.

R. W. Hockney and C. R. Jesshope. Parallel Computers 2: Architecture, Programming and Algorithms. Institute of Physics Publishing, 1988.

T. A. Davis. Performance Analysis with gprof. GNU Project Documentation. https://sourceware.org/binutils/docs/gprof/